

Operating Systems & System Programming

Lesson #15 - Practice

Lesson #15 - Operating Systems & System Programming - Practice



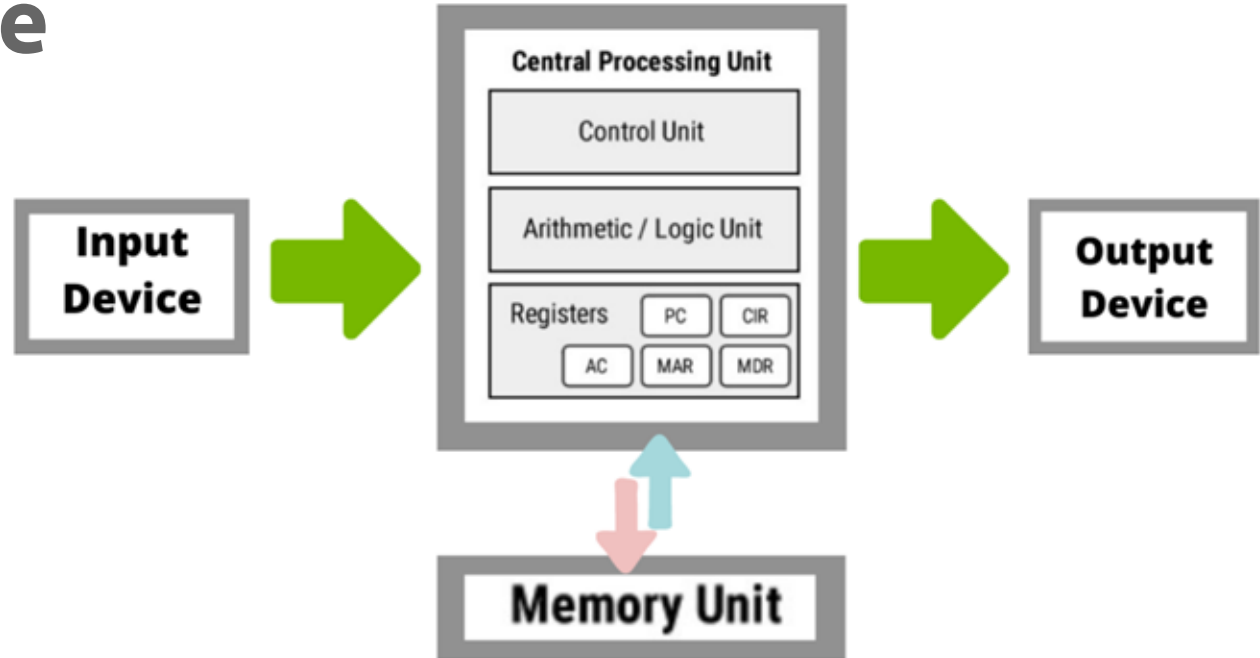
Class work



- Today's practice class objectives
 - Memory Management

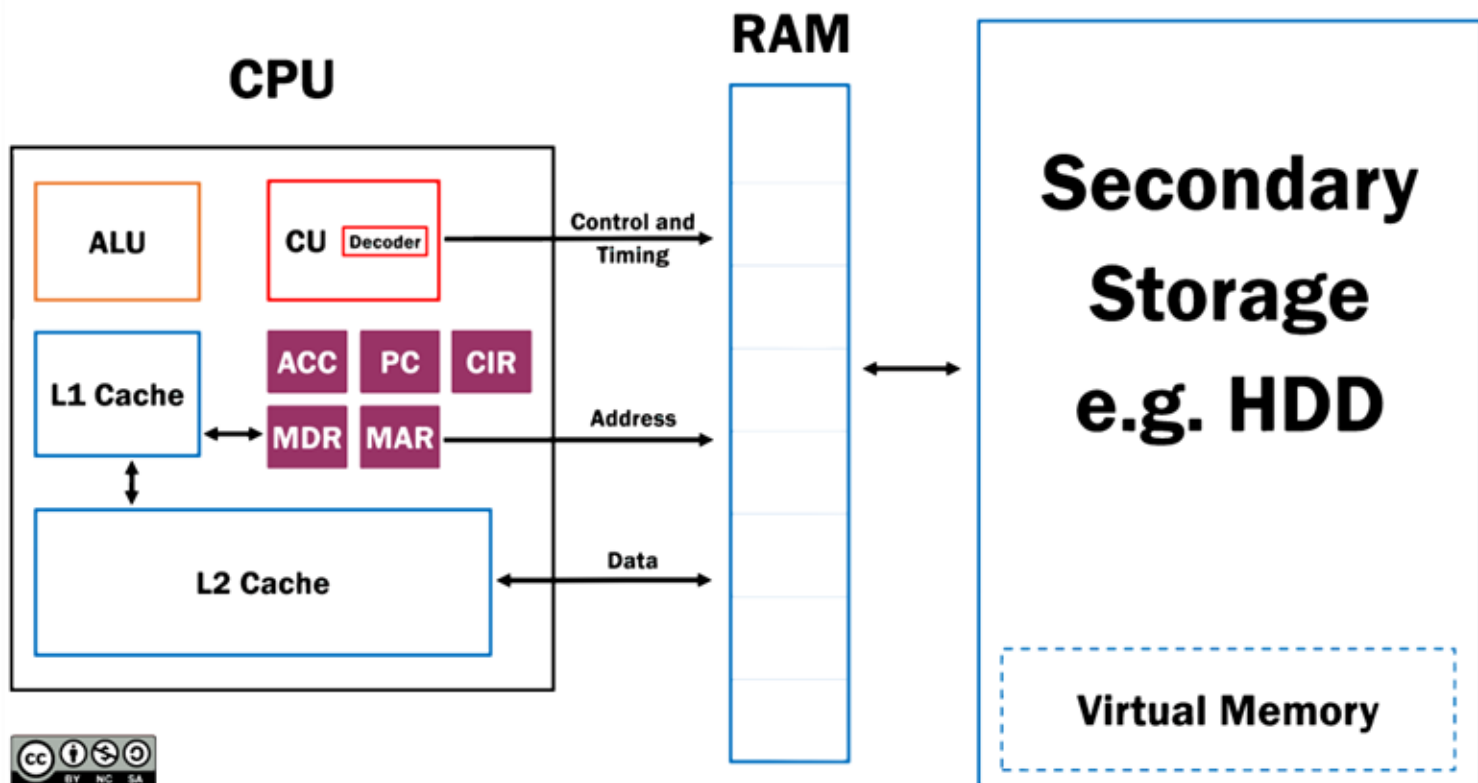
Instruction Set Architecture

Von Neumann Architecture



What you
download to
RAM that you
get

Computer Systems - Von Neumann Architecture

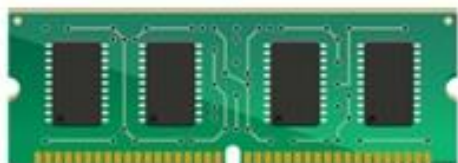


William Lau - Creative Commons - Attribution - Non Commercial - ShareAlike 4.0 International

START



Disk Memory
Secondary Memory
Operating System Loads
Program Into RAM



Primary Memory
Main Memory
RAM



Yes
Service Interrupt

No

1

2

Decode

Fetch

MACHINE
CYCLE

Execute

3

Store

4



Processor CPU

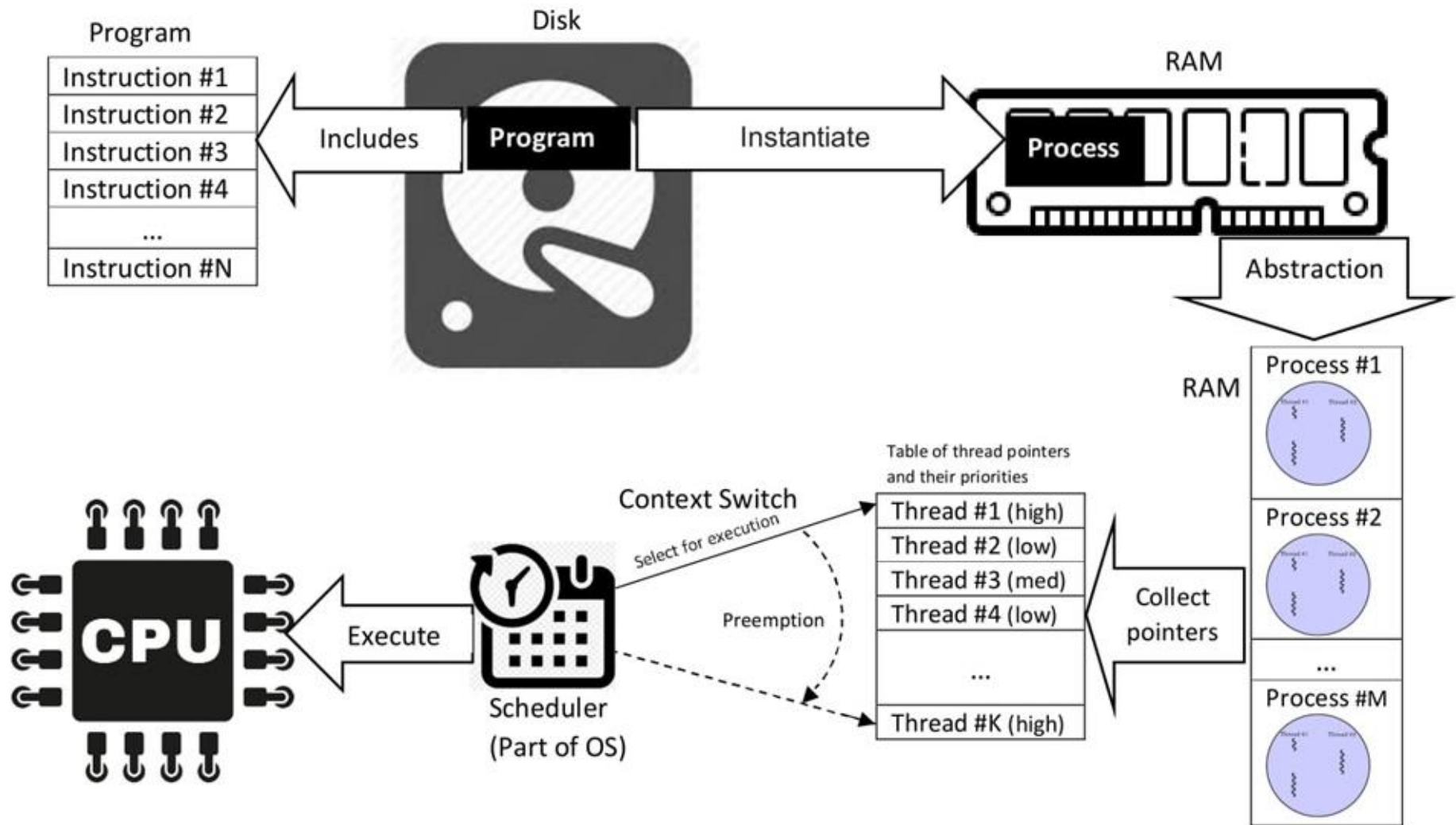


PROGRAM

- 1 Instruction.
- 2 Instruction.
- 3 Instruction.
- 4 Instruction.
- 5 Instruction.
- 6 Instruction.

Program Is Set Of
Instructions Stored In The
Main Memory
RAM





Lesson #15 - Operating Systems & System Programming - Practice

Quick Review — Memory Functions

`malloc(size)`

Allocate size bytes, uninitialized.
Returns void*, cast to type needed.

`calloc(n, size)`

Allocate $n \times \text{size}$ bytes, zero-initialized.
Two-argument form.

`realloc(ptr, size)`

Resize existing heap block.
Use returned pointer — may move.

`free(ptr)`

Release heap memory.
Always set `ptr = NULL` afterwards.

Practice Lesson #15

Exercise 01

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = _____;

    if (_____) {
        printf("Allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("arr[%d] = ", i);
        scanf("%d", &arr[i]);
    }

    printf("You entered: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    _____;
    arr = NULL;
    return 0;
}
```

Practice Lesson #15

Hints & Steps

TODO 1: `malloc(n * sizeof(int))`
Don't forget the cast: `(int*)`

TODO 2: Check `arr == NULL`
before using the pointer

TODO 3: Use `&arr[i]` with `scanf`
to fill each slot

TODO 4: Loop and `printf` each
element

TODO 5: `free(arr)` — release
heap memory when done

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = _____;

    if (_____) {
        printf("Allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("arr[%d] = ", i);
        scanf("%d", &arr[i]);
    }

    printf("You entered: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    _____;
    arr = NULL;
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = (int*) malloc(n * sizeof(int)); // ✓

    if (arr == NULL) {                          // ✓
        printf("Allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("arr[%d] = ", i);
        scanf("%d", &arr[i]);                  // ✓
    }

    printf("You entered: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);                 // ✓
    printf("\n");

    free(arr);                                  // ✓
    arr = NULL;
    return 0;
}
```

Exercise 02

Practice Lesson #15

```
$ ./a.out
Enter number of elements: 4
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
You entered: 10 20 30 40
```

Always use sizeof for portability across platforms.

Accessing a NULL pointer crashes the program.

Two steps: free the heap block, then clear the local pointer.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = (int*) malloc(n * sizeof(int)); // ✓

    if (arr == NULL) { // ✓
        printf("Allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("arr[%d] = ", i);
        scanf("%d", &arr[i]); // ✓
    }

    printf("You entered: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]); // ✓
    printf("\n");

    free(arr); // ✓
    arr = NULL;
    return 0;
}
```

Practice Lesson #15

Exercise 03

Zero-Initialized
Buffer with calloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 6;

    // Allocate & zero-initialize using calloc
    double *data = (double*) calloc(n, sizeof(double));
    if (!data) { return 1; }

    // Prove all elements are 0.0 before assignment
    printf("Initial values (all should be 0.0):\n");
    for (int i = 0; i < n; i++)
        printf("  data[%d] = %.1f\n", i, data[i]);

    // Assign some values
    data[0] = 3.14; data[1] = 2.71;
    data[2] = 1.41; data[3] = 1.73;

    printf("\nAfter assignment:\n");
    for (int i = 0; i < n; i++)
        printf("  data[%d] = %.2f\n", i, data[i]);

    free(data);
    data = NULL;
    return 0;
}
```

Practice Lesson #15

Expected Output

Initial values (all should be 0.0):

```
data[0] = 0.0
data[1] = 0.0
data[2] = 0.0
data[3] = 0.0
data[4] = 0.0
data[5] = 0.0
```

✓ `calloc()` guarantees zero-initialization

✓ Use for buffers, counters, flags when default 0 is correct

✓ More readable: `calloc(n, sizeof(T))` vs `malloc(n * sizeof(T))`

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int n = 6;

    // Allocate & zero-initialize using calloc
    double *data = (double*) calloc(n, sizeof(double));
    if (!data) { return 1; }

    // Prove all elements are 0.0 before assignment
    printf("Initial values (all should be 0.0):\n");
    for (int i = 0; i < n; i++)
        printf("  data[%d] = %.1f\n", i, data[i]);

    // Assign some values
    data[0] = 3.14; data[1] = 2.71;
    data[2] = 1.41; data[3] = 1.73;

    printf("\nAfter assignment:\n");
    for (int i = 0; i < n; i++)
        printf("  data[%d] = %.2f\n", i, data[i]);

    free(data);
    data = NULL;
    return 0;
}
```

Practice Lesson #15

Exercise 04

Dynamic Array
Growth with
realloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int capacity = 3;
    int count = 0;
    int *arr = (int*) malloc(capacity * sizeof(int));
    if (!arr) return 1;

    // Fill initial 3 slots
    for (int i = 0; i < 3; i++) arr[count++] = (i+1) * 5;
    printf("Before realloc (%d elements): ", count);
    for (int i = 0; i < count; i++) printf("%d ", arr[i]);
    printf("\n");

    // Grow to capacity 7
    capacity = 7;
    int *tmp = (int*) realloc(arr, capacity * sizeof(int));
    if (!tmp) { free(arr); return 1; } // safe: don't leak!
    arr = tmp;

    // Fill 4 more slots
    for (int i = 3; i < 7; i++) arr[count++] = (i+1) * 5;
    printf("After realloc (%d elements): ", count);
    for (int i = 0; i < count; i++) printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}
```

Practice Lesson #15

Critical Pattern

✗ WRONG:

```
arr = realloc(arr, ...);
```

If realloc() fails, arr becomes NULL and original block is lost — memory leak!

✓ CORRECT:

```
int *tmp = realloc(arr, ...);  
if (!tmp) { free(arr); return 1; }  
arr = tmp;
```

Use a temporary pointer. On failure, free original and exit safely.

```
#include <stdio.h>  
#include <stdlib.h>
```

```
int main() {  
    int capacity = 3;  
    int count = 0;  
    int *arr = (int*) malloc(capacity * sizeof(int));  
    if (!arr) return 1;  
  
    // Fill initial 3 slots  
    for (int i = 0; i < 3; i++) arr[count++] = (i+1) * 5;  
    printf("Before realloc (%d elements): ", count);  
    for (int i = 0; i < count; i++) printf("%d ", arr[i]);  
    printf("\n");  
  
    // Grow to capacity 7  
    capacity = 7;  
    int *tmp = (int*) realloc(arr, capacity * sizeof(int));  
    if (!tmp) { free(arr); return 1; } // safe: don't leak!  
    arr = tmp;  
  
    // Fill 4 more slots  
    for (int i = 3; i < 7; i++) arr[count++] = (i+1) * 5;  
    printf("After realloc (%d elements): ", count);  
    for (int i = 0; i < count; i++) printf("%d ", arr[i]);  
    printf("\n");  
  
    free(arr);  
    return 0;  
}
```


Practice Lesson #15

Exercise 05

Dynamic Array of Structs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float gpa;
} Student;

int main() {
    int n = 3;

    Student *students = (Student*) malloc(n * sizeof(Student));
    if (!students) return 1;

    students[0] = (Student){1, "Aizat", 3.8f};
    students[1] = (Student){2, "Bekzat", 3.5f};
    students[2] = (Student){3, "Daniya", 3.9f};

    printf("%-5s %-15s %s\n", "ID", "Name", "GPA");
    printf("-----\n");
    for (int i = 0; i < n; i++)
        printf("%-5d %-15s %.1f\n",
            students[i].id, students[i].name, students[i].gpa);

    free(students);
    students = NULL;
    return 0;
}
```

Practice Lesson #15

Expected Output

ID	Name	GPA
1	Aizat	3.8
2	Bekzat	3.5
3	Daniya	3.9

Key Rules

- `malloc(n * sizeof(Student))` — always use the struct's size.
- Access fields with `students[i].name` syntax.
- One `free()` call releases the whole array.
- For nested pointers in structs, each must be freed separately.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
typedef struct {
    int id;
    char name[50];
    float gpa;
} Student;
```

```
int main() {
    int n = 3;
```

```
    Student *students = (Student*) malloc(n * sizeof(Student));
    if (!students) return 1;
```

```
    students[0] = (Student){1, "Aizat", 3.8f};
    students[1] = (Student){2, "Bekzat", 3.5f};
    students[2] = (Student){3, "Daniya", 3.9f};
```

```
    printf("%-5s %-15s %s\n", "ID", "Name", "GPA");
    printf("-----\n");
```

```
    for (int i = 0; i < n; i++)
        printf("%-5d %-15s %.1f\n",
               students[i].id, students[i].name, students[i].gpa);
```

```
    free(students);
    students = NULL;
    return 0;
```

```
}
```

Lesson #15 - Operating Systems & System Programming - Practice

Valgrind — Memory Error Checker

What is Valgrind?

Valgrind is a free instrumentation framework for Linux and macOS that detects:

- Memory leaks (malloc without free)
- Use of uninitialized memory
- Invalid reads / writes
- Double free errors
- Use after free

It runs your program inside a virtual CPU that tracks every memory access.

How to Use

1. Compile with -g flag

```
gcc -g -o program program.c
```

2. Run under Valgrind

```
valgrind --leak-check=full \  
./program
```

3. Read the report

```
LEAK SUMMARY:  
    definitely lost: 400 bytes  
ERROR SUMMARY: 1 error
```

Lesson #15 - Operating Systems & System Programming - Practice

Classwork results

Make screenshots of your code and output (terminal).

P1 Dynamic integer array: malloc(), NULL check, user input, print, free().

P2 Zero-initialized buffer: calloc() for a double array; prove values are 0 before assignment.

P3 Grow an array: malloc(3), fill, then realloc(7) and fill 4 more elements.

P4 Struct array: Dynamic array of Student structs; print formatted table.

P5 Find & fix the leak (Bonus): Fix the buggy code from Slide 10; optionally confirm with Valgrind.

Lesson #15 - Operating Systems & System Programming - Practice

Q & A